

Rising Beyond Borders

Document 20 — Security & Production Hardening

Master Specification + Claude Code Implementation Prompt

Purpose: harden the existing RBB website for production without changing its information architecture, adding a CMS, inventing security claims, or introducing new third-party services. This is an engineering hardening phase, not a security certification or penetration-test report.

Current baseline: Documents 01–19 established the site architecture, approved-content publishing gate, pre-rendering, accessibility engineering target, observability framework, and content-readiness controls. Document 20 turns those controls into a deliberate production security baseline.

1. Scope

- Browser security headers and policy strategy.
- Content Security Policy design and rollout without breaking the current site.
- Transport security and HTTPS assumptions.
- Clickjacking, MIME sniffing and referrer-policy protections.
- Permissions Policy review.
- Secret and environment-variable hygiene.
- Dependency and supply-chain review.
- Build-output inspection.
- Client-side exposure review.
- Third-party origin review.
- Form/payment/provider boundary protection for future phases.
- Error handling and information disclosure.
- Source maps and production artifact exposure.
- Static hosting/server configuration requirements.
- Security regression tests and launch checklist.

2. Non-Goals

- No CMS or admin system.
- No authentication system.
- No backend implementation in this phase.
- No payment integration.
- No form submission provider.
- No analytics provider.
- No security certification claim.
- No claim that the site is penetration-tested.
- No invented hosting-provider security features.
- No production secrets added to the repository.

3. Security Baseline

Control	Target	Requirement
HTTPS	Required in production	Production origin must be HTTPS; HTTP→HTTPS enforcement belongs at the host/CDN.
HSTS	Production only	Enable only after HTTPS is confirmed across the complete production domain.
Content-Security-Policy	Required	Use a restrictive policy derived from actual runtime origins; avoid unsafe broad allowances.
Frame protection	Required	Use CSP frame-ancestors and/or X-Frame-Options as appropriate for browser compatibility.
MIME sniffing	Required	Send X-Content-Type-Options: nosniff.
Referrer Policy	Required	Use a privacy-conscious policy compatible with site requirements.
Permissions Policy	Required/reviewed	Disable capabilities the site does not need.
Secrets	No client secrets	Only public browser-safe configuration may ship to the client.
Source maps	Controlled	Do not expose source maps publicly unless deliberately approved.
Dependencies	Reviewed	Check direct/transitive dependencies and lockfile changes.

4. Content Security Policy

CSP must be derived from the actual application rather than copied from a generic template. At the current baseline, the site is static/pre-rendered and has no analytics, payments, forms, maps, video embeds or other application providers. The known third-party font origins from the technology audit must be explicitly considered.

- Start with report-only testing if necessary to discover violations without breaking production.
- Allow only origins genuinely required by the built site.
- Do not use broad wildcards such as https: or * unless a specific documented requirement exists.
- Do not add unsafe-inline or unsafe-eval merely to make a violation disappear.
- If inline styles/scripts are required by the current build, prefer nonces/hashes or an architecture change over broad unsafe allowances.
- Ensure the policy works with the pre-rendered HTML and hydrated React application.
- Re-test navigation, mobile menu, fonts, images, redirects and all 28 current pages after enforcement.
- Document any unavoidable exception and its reason.

5. Third-Party Origin Inventory

- Existing technology audit identified Fontshare/Google font-related origins including api.fontshare.com, cdn.fontshare.com, fonts.googleapis.com and fonts.gstatic.com.
- Do not assume every listed origin is currently required; verify actual network requests in the production build.
- Do not add new third-party origins during this phase.
- Future forms, payments, analytics, monitoring or other providers must be added through their own approved integration phase and reflected in the CSP/origin inventory.
- Production host/CDN origin is currently not selected; keep host-specific headers documented rather than inventing provider syntax.

6. Security Headers

- **Strict-Transport-Security:** production HTTPS only; choose max-age deliberately and do not enable includeSubDomains/preload without domain-wide verification.
- **Content-Security-Policy:** enforce after report-only validation.

- **X-Content-Type-Options:** nosniff.
- **Referrer-Policy:** strict-origin-when-cross-origin or another explicitly justified privacy-conscious policy.
- **Permissions-Policy:** deny unused camera, microphone, geolocation and other sensitive capabilities.
- **Frame protections:** protect pages from unauthorized framing unless an approved integration requires embedding.
- **Cross-Origin-Opener-Policy:** evaluate based on actual third-party integrations; do not enable blindly.
- **Cross-Origin-Resource-Policy:** evaluate for static assets and font behavior before enforcing.
- **Cache-Control:** immutable caching for fingerprinted assets where supported; do not cache future personalized/payment responses publicly.

7. Environment & Secret Hygiene

- Review all VITE_* variables and classify each as public configuration or secret.
- Anything exposed through Vite/browser JavaScript is public and must never be treated as a secret.
- Production `.env` files containing secrets must not be committed.
- Do not put payment keys, email credentials, API tokens, private keys or webhook secrets in frontend content/configuration.
- Keep `.env.example` free of real credentials.
- Search the repository and build output for common credential patterns before launch.
- Check generated HTML/JS/CSS for accidental email addresses, internal URLs, tokens, test credentials or private paths.
- Do not store sensitive unpublished RBB content in `src/content/`.

8. Dependency & Supply-Chain Review

- Inspect package.json and the lockfile.
- Run the repository's supported dependency audit command and record findings.
- Do not automatically upgrade packages solely to clear every warning; assess compatibility and scope.
- Identify abandoned/high-risk dependencies and whether they are actually required.
- Confirm no development-only test dependency is accidentally shipped to production.
- Check for unexpected postinstall/build scripts before introducing new packages.
- Keep the current dependency surface stable unless a security fix is necessary.

9. Build Artifact & Information Disclosure Review

- Inspect the final dist directory.
- Verify held-back content remains absent.
- Verify no `.env`, credentials, private configuration or repository metadata is copied to dist.
- Verify no test fixtures or temporary content are shipped.
- Review source maps and decide whether they are disabled, private or deliberately public.
- Check generated error/404 pages for internal stack traces or filesystem paths.
- Ensure ErrorBoundary and 404 behavior expose generic user-facing messages without sensitive implementation details.
- Check response headers and static MIME types where the hosting platform permits configuration.

10. Client-Side Attack Surface

- Review all user-controlled URL/query parameters and ensure they are not injected into HTML unsafely.
- Review external links and use appropriate target/rel behavior where new tabs are used.
- Check any future `dangerouslySetInnerHTML` usage; it should not be introduced for normal RBB content.
- Ensure content files cannot inject executable HTML or script into rendered pages.
- Keep the current approved-content validation boundary.
- Verify route handling does not interpret arbitrary paths as HTML templates.

- Ensure error states cannot reflect attacker-controlled content.

11. Future Forms & Payments Security Boundary

Documents 21 and 22 will handle real form/email and donation/payment integrations. Document 20 only establishes the boundary.

- All validation must be server-side once submissions exist.
- Rate limiting and anti-spam belong at the submission boundary.
- Payment secrets and webhook verification must remain server-side/provider-side.
- Never trust payment success based only on client redirect/query parameters.
- Never log full payment details, tokens, passwords or sensitive personal data.
- Any new provider must trigger an origin/CSP/privacy/technology-audit update.

12. Security Testing Matrix

Test	Expected result
Production build	Build succeeds with no warnings introduced by hardening.
Header inspection	Required headers present on production responses once host is configured.
CSP report-only/enforcement	No unexpected blocked resources; no unsafe broad allowances.
HTTPS	No mixed-content resources; HTTP redirects to HTTPS at host.
Secret scan	No credentials/secrets in source or dist.
Dependency audit	Findings documented; no unexplained high-risk dependency introduced.
Dist inspection	No private files, test fixtures or held-back content.
Source-map review	Production policy documented and verified.
Route fuzzing	Unknown routes return real 404 behavior without sensitive output.
Query parameter review	No reflected HTML/script injection.
Accessibility regression	Existing accessibility checks remain clean.
Hydration	No hydration mismatch or security-related console errors.
Technology audit	Third-party origins match the approved inventory.

13. Production Host Requirements

- Production domain must use HTTPS.
- Permanent HTTP redirects should be configured at the host/CDN, not only through React.
- Security headers should be delivered as HTTP response headers where possible.
- Host/CDN should be audited for cookies, injected scripts, analytics, challenge pages and other platform behavior.
- Static asset caching should preserve content-hash semantics.
- 404 responses should use a genuine HTTP 404 status.
- Any host-specific configuration must be documented in `docs/DEPLOYMENT.md`.
- Do not claim a host security feature until the selected production host has been verified.

14. Security Documentation

- Create/update `docs/SECURITY.md` with the implemented baseline and remaining dependencies.
- Record actual headers and CSP decisions.
- Record dependency audit date and findings.
- Record secret-scan result.
- Record production artifact inspection result.
- Record third-party origins.
- Clearly distinguish implemented controls from controls that require the final host.
- Do not publish unsupported claims such as 'fully secure', 'SOC 2 compliant', 'penetration tested' or 'certified'.

15. Claude Code Implementation Prompt

You are implementing Document 20 – Security & Production Hardening for the Rising Beyond Borders website.

GOAL

Harden the existing static/pre-rendered React site for production without changing its architecture or adding a backend, CMS, payment provider, analytics provider, or authentication system.

CURRENT BASELINE

- 28 production-generated pages plus 404 after Document 19.
- Pre-rendered HTML with React hydration.
- Approved-content publishing gate using editorialStatus.
- No analytics, payment, form submission provider or monitoring provider enabled.
- Existing technology audit identifies font-related third-party origins.
- Production host/domain has not yet been selected.

NON-NEGOTIABLE

- Do not invent a hosting provider.
- Do not invent security certification/compliance.
- Do not add secrets.
- Do not add third-party services.
- Do not redesign.
- Do not weaken the approved-content filter.
- Do not introduce unsafe-inline or unsafe-eval just to silence CSP violations.
- Do not stage or commit changes.

IMPLEMENT

1. Inspect current configuration, Vite build, pre-rendering, env handling, observability, technology audit and deployment docs.
2. Inventory actual runtime third-party origins from the production build.
3. Design a restrictive CSP from actual requirements.
4. Use report-only first if needed, then enforce only after violations are understood.
5. Add/prepare security headers: CSP, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, frame protections and other appropriate headers.
6. Treat HSTS as production-host dependent; do not pretend it is active locally.
7. Review environment variables. Anything shipped to browser is public.
8. Scan source and build output for secrets, tokens, credentials, private URLs and accidental sensitive data.
9. Inspect package.json/lockfile and run the supported dependency audit.
10. Do not perform broad dependency upgrades unless required for a security issue.
11. Inspect dist for .env files, test fixtures, private content, repository metadata and accidental source maps.
12. Review ErrorBoundary/404 output for information disclosure.
13. Review query/path handling for reflected injection or unsafe HTML.
14. Ensure RBB content cannot inject executable markup.
15. Preserve all existing content, SEO, accessibility, hydration, redirect and route behavior.
16. Update docs/SECURITY.md with implemented controls and unresolved host-dependent controls.
17. Update docs/TECHNOLOGY_AUDIT.md if the origin inventory changes.
18. Run build, content validation, route tests, accessibility tests, hydration tests and technology audit.

SECURITY TESTS

- Build passes with no warnings.
- Required headers are represented correctly in the configured deployment mechanism.
- CSP has no unexplained broad wildcard.
- No credentials in source or dist.
- No held-back content in dist.
- No private files in dist.
- No unexpected third-party origins.

- Unknown routes remain real 404s.
- Existing redirects remain intact.
- No accessibility regression.
- No hydration regression.
- No console errors.

REPORT

Return:

- files changed
- security headers implemented
- CSP policy and any exceptions
- third-party origins discovered
- dependency audit result
- secret-scan result
- dist inspection result
- source-map decision
- host-dependent items still pending
- tests run and results
- any unresolved security concern
- whether anything was staged or committed

Do not claim the site is secure in an absolute sense. Report the concrete controls and remaining dependencies.

16. Acceptance Criteria

- A documented security baseline exists.
- CSP is based on actual site requirements and does not use unnecessary broad allowances.
- Security headers are implemented or explicitly marked host-dependent.
- No secrets are present in source or production output.
- Dependency/security findings are documented.
- Production artifacts contain no private/test/held-back material.
- Error pages do not disclose implementation details.
- Third-party origins are inventoried.
- Existing content, SEO, accessibility, redirects and hydration remain intact.
- No new external provider was introduced.
- Security documentation distinguishes implemented controls from future host-dependent controls.

Document status: Implementation specification. Production security posture remains dependent on the final hosting/domain configuration and any future form/payment/provider integrations.